

Task 05 — Code Walkthrough

Line-by-Line Reference: Classes, Triggers, Async Apex, Testing, and API Mechanics

Companion to the full Technical Report — this document covers ONLY how the code works

Prepared by: Ajher

How to Read This Document

This is a pure code-mechanics reference. It does not cover the business requirements, the platform configuration screens, or the deliverable evidence — those live in the companion Technical Report. Here, every class is walked through block by block: what each part does, why it is written that way, and exactly how control and data pass from one piece to the next.

Boxes labeled "Execution Trace" describe the runtime order of events — what happens first, second, third — across method and class boundaries, since that ordering is often the hardest part of an integration like this to hold in your head from reading the source files alone.

1. Execution Order — The Big Picture

There are exactly two ways this system starts processing an Opportunity, and both of them funnel into the same one method — `CurrencyConversionService.processOpportunities()` — after that point, execution is identical regardless of which path started it.

1.1 Path A — Bulk / Manual Batch Run

- 1. A developer or admin runs: `Database.executeBatch(new CurrencyConversionBatch(), 200);`
- 2. Salesforce calls `CurrencyConversionBatch.start()` ONCE, which returns a `QueryLocator` over every Opportunity (or a filtered subset, if a `WHERE` clause was passed to the constructor).
- 3. Salesforce splits the query results into chunks of up to 200 records and calls `CurrencyConversionBatch.execute()` ONCE PER CHUNK, each in its own separate transaction.
- 4. Each `execute()` call passes its 200-record chunk straight into `CurrencyConversionService.processOpportunities(scope)`.
- 5. `processOpportunities()` does all the real work (Section 3) and returns a `ConversionOutcome` summary back to `execute()`, which adds it to the batch's running totals.
- 6. After every chunk has run, Salesforce calls `CurrencyConversionBatch.finish()` ONCE, which logs the final totals.

1.2 Path B — Real-Time, Triggered by a Record Change

- 1. A user (or the Data Import tool) inserts or updates an Opportunity, setting or changing Amount and/or Expected_Currency_Type__c.
- 2. The database commit fires `OpportunityCurrencyTrigger`, an after insert / after update trigger.
- 3. The trigger immediately calls `OpportunityCurrencyTriggerHandler.handle(Trigger.new, Trigger.oldMap)` and does nothing else itself.
- 4. `handle()` decides, per record, whether this change actually needs processing (Section 4.2), and if so, collects the qualifying Ids into a `Set<Id>`.
- 5. If that set is non-empty, `handle()` calls `System.enqueueJob(new CurrencyConversionQueueable(idsToProcess))` exactly once for the whole transaction, then returns — the trigger transaction commits and ends here.
- 6. Separately and asynchronously (usually within seconds, but not synchronously with step 5), Salesforce executes `CurrencyConversionQueueable.execute()`, which re-queries the affected Opportunities fresh from the database.
- 7. The Queueable passes that re-queried list into the SAME `CurrencyConversionService.processOpportunities()` method used by Path A.

Execution Trace — Why both paths converge on one method

Notice that steps 4-5 of Path A and step 7 of Path B both end at the identical call: `CurrencyConversionService.processOpportunities(someListOfOpportunities)`. Everything upstream of that call is just "how did we decide which records to process, and how did we get permission to make a callout" — the actual grouping, validation, calculation, and DML logic exists in exactly one place in the whole codebase. This is deliberate: it is architecturally impossible for the bulk path and the real-time path to disagree about what counts as a valid conversion, because there is only one implementation of that logic to disagree with itself.

2. ExchangeRateApiClient — Line by Line

This class has one public method, `getConversionRate()`, and one job: turn a base currency and a target currency into a typed result describing whether the API call succeeded, and if so, what rate it returned.

```
public with sharing class ExchangeRateApiClient {

    public class RateResult {
        public Boolean isSuccess;
        public Decimal conversionRate;
        public String errorType;
        public String errorMessage;
        public Integer httpStatusCode;
    }

    public class ExchangeRateApiException extends Exception {}

    private static final String NAMED_CREDENTIAL = 'callout:ExchangeRateAPI';

    public static RateResult getConversionRate(String baseCurrency, String targetCurrency) {
        RateResult result = new RateResult();

        if (String.isBlank(baseCurrency) || String.isBlank(targetCurrency)) {
            result.isSuccess = false;
            result.errorType = 'invalid-input';
            result.errorMessage = 'Base or target currency code is blank.';
            return result;
        }

        HttpRequest req = new HttpRequest();
        req.setEndpoint(NAMED_CREDENTIAL + '/pair/' + baseCurrency.trim() + '/' +
targetCurrency.trim());
        req.setMethod('GET');
        req.setTimeout(10000);

        Http http = new Http();
        HttpResponse res;

        try {
            res = http.send(req);
        } catch (System.CalloutException ce) {
            result.isSuccess = false;
            result.errorType = 'callout-exception';
            result.errorMessage = ce.getMessage();
            return result;
        }

        result.httpStatusCode = res.getStatusCode();
    }
}
```

```

    Map<String, Object> body;
    try {
        body = (Map<String, Object>) JSON.deserializeUntyped(res.getBody());
    } catch (Exception je) {
        result.isSuccess = false;
        result.errorType = 'parse-error';
        result.errorMessage = 'Could not parse API response: ' + je.getMessage();
        return result;
    }

    String apiResult = (String) body.get('result');

    if (res.getStatusCode() == 200 && apiResult == 'success') {
        result.isSuccess = true;
        Object rateObj = body.get('conversion_rate');
        result.conversionRate = (rateObj != null) ?
Decimal.valueOf(String.valueOf(rateObj)) : null;
    } else {
        result.isSuccess = false;
        result.errorType = (String) body.get('error-type');
        result.errorMessage = String.isNotBlank(result.errorType)
            ? 'API error: ' + result.errorType
            : 'Unexpected response, HTTP ' + res.getStatusCode();
    }

    return result;
}
}

```

2.1 The RateResult inner class

`public class RateResult { ... }` is a plain data container — five public fields, no methods, no constructor logic. It exists purely so `getConversionRate()` can return several pieces of information at once (success flag, the rate itself, an error type, a human-readable error message, and the raw HTTP status code) as a single object, rather than needing five separate out-parameters or a generic `Map` that callers would have to cast values out of.

2.2 The blank-input guard (lines near the top of `getConversionRate`)

`if (String.isBlank(baseCurrency) || String.isBlank(targetCurrency)) { ... return result; }` runs BEFORE anything else in the method — before an `HttpRequest` is even constructed. If either currency code is blank, the method immediately builds a failed `RateResult` with `errorType = 'invalid-input'` and returns it. No network activity happens at all for this case. This is why the corresponding test (`testBlankCurrencyInput`) needs no `HttpCalloutMock` — the method physically cannot reach the point where a callout would occur.

2.3 Building and sending the request

- `HttpRequest req = new HttpRequest();` creates an empty request object.

- `req.setEndpoint(NAMED_CREDENTIAL + '/pair/' + baseCurrency.trim() + '/' + targetCurrency.trim());` builds the final URL. `NAMED_CREDENTIAL` is the constant `'callout:ExchangeRateAPI'` — everything from that point onward in the string is just a URL PATH appended after the Named Credential resolves its real hostname. For BDT -> USD, this produces `callout:ExchangeRateAPI/pair/BDT/USD`.
- `req.setMethod('GET');` — a GET request, since this is a read-only lookup with no request body.
- `req.setTimeout(10000);` — caps how long Apex will wait for a response, in milliseconds (10 seconds here). If the API hangs longer than this, Apex raises a `CalloutException` on its own rather than blocking indefinitely.
- `Http http = new Http(); HTTPResponse res = http.send(req);` actually sends the request over the network and blocks until a response arrives or the timeout elapses.

2.4 The try/catch around the send

The `http.send(req)` call is wrapped in try/catch (`System.CalloutException ce`). This specific exception type is what Apex throws for transport-level failures — DNS resolution failure, connection refused, the timeout from 2.3 being exceeded, and similar. Catching it here and converting it into a failed `RateResult` (`errorType = 'callout-exception'`) means a genuine network problem is reported back to the caller in exactly the same shape as an API-level rejection, so `CurrencyConversionService` never needs to know or care which kind of failure occurred — it only ever inspects `result.isSuccess`.

2.5 Parsing the response body

- `result.httpStatusCode = res.getStatusCode();` is recorded unconditionally, for both success and failure paths.
- `Map<String, Object> body = (Map<String, Object>) JSON.deserializeUntyped(res.getBody());` turns the raw JSON text into a generic, navigable structure. This line is itself wrapped in its own try/catch (`Exception je`) — if the response body is not valid JSON at all (tested explicitly via `MalformedJsonMock` in the test suite), this cast/parse throws, and that catch block converts it into a failed `RateResult` with `errorType = 'parse-error'`, again in the same shape as every other failure mode.
- `String apiResult = (String) body.get('result');` reads the top-level "result" key that `ExchangeRate-API` always includes — its value is literally the string "success" or "error".

2.6 The success/failure branch

`if (res.getStatusCode() == 200 && apiResult == 'success')` checks BOTH the HTTP status code and the API's own self-reported result field — belt and suspenders, since a well-behaved API should never send HTTP 200 with an internal "error" body, but checking both costs nothing and protects against an API that does something unexpected.

On success: `Object rateObj = body.get('conversion_rate');` pulls the numeric rate out of the map as a generic `Object` (because that is what `deserializeUntyped` always returns for JSON numbers — internally a `Double`). `result.conversionRate = Decimal.valueOf(String.valueOf(rateObj));` converts it safely into a `Decimal` by round-tripping through a `String`, because casting a `Double` directly to `Decimal`, (`Decimal`) `rateObj`, throws a runtime exception in Apex — this `String` round-trip is the standard, necessary workaround.

On failure: `result.errorType = (String) body.get('error-type');` reads the specific error code the API provides (e.g. 'unsupported-code', 'invalid-key'), and `result.errorMessage` is built from it, falling back to a generic message mentioning the HTTP status code if the API didn't include an error-type at all.

3. CurrencyConversionService — Line by Line

This is the largest and most important class in the project — every decision about what counts as success or failure, and every write to the database, happens here. It is called identically from the batch path and the trigger/Queueable path (Section 1).

```
public with sharing class CurrencyConversionService {

    private static final String BASE_CURRENCY = 'BDT';
    private static final Set<String> SUPPORTED_CURRENCIES = new Set<String>{
        'USD', 'EUR', 'GBP', 'JPY', 'AUD', 'CAD'
    };

    public class ConversionOutcome {
        public Integer successCount = 0;
        public Integer failedCount = 0;
        public Integer skippedCount = 0;
        public Integer apiCallCount = 0;
    }

    public class InsufficientAccessException extends Exception {}

    private static void enforceFieldLevelSecurity() {
        Boolean canReadOpportunityFields =
            Schema.sObjectType.Opportunity.fields.Amount.isAccessible() &&
            Schema.sObjectType.Opportunity.fields.Expected_Currency_Type__c.isAccessible() &&
            Schema.sObjectType.Opportunity.fields.Expected_Currency_Amount__c.isAccessible();

        Boolean canUpdateOpportunityFields =
            Schema.sObjectType.Opportunity.fields.Expected_Currency_Amount__c.isUpdateable() &&
            Schema.sObjectType.Opportunity.fields.Exchange_Rate__c.isUpdateable() &&
            Schema.sObjectType.Opportunity.fields.Currency_Last_Updated__c.isUpdateable() &&
            Schema.sObjectType.Opportunity.fields.Currency_Update_Status__c.isUpdateable() &&
            Schema.sObjectType.Opportunity.fields.Currency_Error_Message__c.isUpdateable();

        Boolean canCreateHistoryRecords =
            Schema.sObjectType.Exchange_Rate_History__c.isCreateable();
        Boolean canCreateTrackingRecords =
            Schema.sObjectType.Opportunity_Currency_Update__c.isCreateable();

        if (!canReadOpportunityFields || !canUpdateOpportunityFields ||
            !canCreateHistoryRecords || !canCreateTrackingRecords) {
            throw new InsufficientAccessException(
                'The running user lacks the field-level or object-level permissions required for
' +
                'currency conversion processing. Verify the "Currency Integration Access"
permission ' +
                'set is assigned.');
```

```

    }

    public static ConversionOutcome processOpportunities(List<Opportunity> opps) {
        enforceFieldLevelSecurity();

        ConversionOutcome outcome = new ConversionOutcome();

        Map<String, List<Opportunity>> byCurrency = new Map<String, List<Opportunity>>();

        for (Opportunity opp : opps) {
            String currencyCode = opp.Expected_Currency_Type__c;

            if (String.isBlank(currencyCode)) {
                outcome.skippedCount++;
                continue;
            }

            currencyCode = currencyCode.trim().toUpperCase();
            if (!byCurrency.containsKey(currencyCode)) {
                byCurrency.put(currencyCode, new List<Opportunity>());
            }
            byCurrency.get(currencyCode).add(opp);
        }

        Map<String, ExchangeRateApiClient.RateResult> rateCache = new Map<String,
ExchangeRateApiClient.RateResult>();
        for (String currencyCode : byCurrency.keySet()) {
            if (SUPPORTED_CURRENCIES.contains(currencyCode)) {
                ExchangeRateApiClient.RateResult rate =
ExchangeRateApiClient.getConversionRate(BASE_CURRENCY, currencyCode);
                rateCache.put(currencyCode, rate);
                outcome.apiCallCount++;
            }
        }

        List<Opportunity> oppsToUpdate = new List<Opportunity>();
        List<Exchange_Rate_History__c> rateHistoryRecords = new List<Exchange_Rate_History__c>();
        List<Opportunity_Currency_Update__c> currencyUpdateRecords = new
List<Opportunity_Currency_Update__c>();
        DateTime nowTs = DateTime.now();

        for (String currencyCode : byCurrency.keySet()) {
            ExchangeRateApiClient.RateResult rate = rateCache.get(currencyCode);

            if (SUPPORTED_CURRENCIES.contains(currencyCode)) {
                Exchange_Rate_History__c historyRec = new Exchange_Rate_History__c();
                historyRec.Base_Currency__c = BASE_CURRENCY;
                historyRec.Target_Currency__c = currencyCode;
                historyRec.Retrieved_Date_Time__c = nowTs;
            }
        }
    }

```

```

        if (rate.isSuccess) {
            historyRec.API_Status__c = 'Success';
            historyRec.Exchange_Rate__c = rate.conversionRate;
        } else {
            historyRec.API_Status__c = 'Failed';
            historyRec.Error_Details__c = rate.errorMessage;
        }
        rateHistoryRecords.add(historyRec);
    }

    for (Opportunity opp : byCurrency.get(currencyCode)) {
        Opportunity oppUpdate = new Opportunity(Id = opp.Id);
        Decimal previousAmount = opp.Expected_Currency_Amount__c;

        if (opp.Amount == null) {
            oppUpdate.Currency_Update_Status__c = 'Failed';
            oppUpdate.Currency_Error_Message__c = 'Opportunity Amount is blank.';
            outcome.failedCount++;

        } else if (!SUPPORTED_CURRENCIES.contains(currencyCode)) {
            oppUpdate.Currency_Update_Status__c = 'Failed';
            oppUpdate.Currency_Error_Message__c = 'Unsupported currency: ' + currencyCode
+
            ' . Supported currencies are USD, EUR, GBP, JPY, AUD, CAD.';
            outcome.failedCount++;

        } else if (!rate.isSuccess) {
            oppUpdate.Currency_Update_Status__c = 'Failed';
            oppUpdate.Currency_Error_Message__c = rate.errorMessage;
            outcome.failedCount++;

        } else {
            Decimal newAmount = (opp.Amount * rate.conversionRate).setScale(2,
System.RoundingMode.HALF_UP);
            oppUpdate.Expected_Currency_Amount__c = newAmount;
            oppUpdate.Exchange_Rate__c = rate.conversionRate;
            oppUpdate.Currency_Last_Updated__c = nowTs;
            oppUpdate.Currency_Update_Status__c = 'Success';
            oppUpdate.Currency_Error_Message__c = null;
            outcome.successCount++;

            Opportunity_Currency_Update__c trackingRec = new
Opportunity_Currency_Update__c();
            trackingRec.Opportunity__c = opp.Id;
            trackingRec.Opportunity_Amount_BDT__c = opp.Amount;
            trackingRec.Expected_Currency_Type__c = currencyCode;
            trackingRec.Exchange_Rate_Used__c = rate.conversionRate;
            trackingRec.Previous_Expected_Currency_Amount__c = previousAmount;

```

```

        trackingRec.New_Expected_Currency_Amount__c = newAmount;
        trackingRec.Updated_Date_Time__c = nowTs;
        currencyUpdateRecords.add(trackingRec);
    }

    oppsToUpdate.add(oppUpdate);
}

if (!rateHistoryRecords.isEmpty()) {
    Database.insert(rateHistoryRecords, false);
}
if (!oppsToUpdate.isEmpty()) {
    Database.update(oppsToUpdate, false);
}
if (!currencyUpdateRecords.isEmpty()) {
    Database.insert(currencyUpdateRecords, false);
}

return outcome;
}
}

```

3.1 Class-level constants

- `BASE_CURRENCY = 'BDT'` — a named constant rather than a repeated string literal, so the base currency only needs to change in one place if it ever did.
- `SUPPORTED_CURRENCIES` — a `Set<String>`, not a `List`. A `Set` gives $O(1)$ membership testing via `.contains()`, which matters because this check runs for every single `Opportunity` being processed; a `List` would require scanning every element on every check.

3.2 The `ConversionOutcome` inner class

A plain counter object with four public `Integer` fields, all initialized to 0. `processOpportunities()` returns one of these per call, so whatever called it (a single batch chunk, or one `Queueable` execution) can report how many records succeeded, failed, were skipped, or how many API calls were actually made — used by `CurrencyConversionBatch` to build a running total across chunks (Section 5).

3.3 `InsufficientAccessException` and `enforceFieldLevelSecurity()`

`InsufficientAccessException` extends `Exception {}` is a one-line custom exception type — giving it its own name (rather than throwing a generic `Exception`) means calling code, or a stack trace, can immediately identify this specific failure mode by type.

`enforceFieldLevelSecurity()` is a private static void method — private because nothing outside this class should ever call it directly; static because it needs no instance state, only the ambient "currently running user" context that `Schema` describe calls automatically read from.

- `canReadOpportunityFields` checks `.isAccessible()` on the three `Opportunity` fields the rest of the method `READS`: `Amount`, `Expected_Currency_Type__c`, `Expected_Currency_Amount__c`.

- `canUpdateOpportunityFields` checks `.isUpdateable()` on the five Opportunity fields the method WRITES: `Expected_Currency_Amount__c`, `Exchange_Rate__c`, `Currency_Last_Updated__c`, `Currency_Update_Status__c`, `Currency_Error_Message__c`.
- `canCreateHistoryRecords` and `canCreateTrackingRecords` check `.isCreateable()` at the OBJECT level (not per-field) for `Exchange_Rate_History__c` and `Opportunity_Currency_Update__c`, since the method only ever inserts whole new records of these types, never updates existing ones.
- The final if statement ORs all four Boolean groups together with negation: if ANY of them is false, the method throws immediately, before a single line of business logic runs. The exception message explicitly names the Permission Set to check, so whoever sees this error in a debug log has an immediate next step.

3.4 The start of `processOpportunities()` — grouping by currency

The very first line of the method body is `enforceFieldLevelSecurity()`; If it throws, nothing below it ever executes — no callout, no DML, nothing.

`Map<String, List<Opportunity>> byCurrency = new Map<String, List<Opportunity>>();` is the grouping structure described in the execution-order overview. The `for (Opportunity opp : opps)` loop that follows does no SOQL and no DML — it only reads the in-memory `opp.Expected_Currency_Type__c` field and writes into this Map, which is why this loop is safe regardless of how many records `opps` contains.

- `String currencyCode = opp.Expected_Currency_Type__c;` reads the raw value.
- `if (String.isBlank(currencyCode)) { outcome.skippedCount++; continue; }` — a blank currency short-circuits immediately: the skipped counter increments, and `continue` jumps straight to the next loop iteration, meaning this Opportunity is added to NO list at all and will never be looked at again for the rest of this method call.
- `currencyCode = currencyCode.trim().toUpperCase();` normalizes whatever survives the blank check — removing accidental whitespace and forcing consistent casing, so "usd", "USD ", and "USD" are all treated identically as the same map key.
- The `if (!byCurrency.containsKey(currencyCode))` block lazily creates a new empty list the first time a given currency is seen, then `byCurrency.get(currencyCode).add(opp);` appends the current Opportunity to whichever list corresponds to its currency.

Execution Trace — State of the data after this first loop

By the time this loop finishes, `byCurrency` might look like: `{"USD": [opp1, opp5, opp9], "EUR": [opp2, opp7], "SGD": [opp3]}` — and every Opportunity that had a blank currency has already been fully accounted for (counted as skipped) and discarded from further processing. Nothing has touched the network or the database yet.

3.5 The callout loop — one request per distinct SUPPORTED currency

`Map<String, ExchangeRateApiClient.RateResult> rateCache = new Map<...>();` will hold at most one entry per distinct currency key from `byCurrency`. The loop `for (String currencyCode : byCurrency.keySet())` iterates the currency KEYS, not the Opportunity lists — so this loop runs once per distinct currency present in the whole input, regardless of how many Opportunities share that currency.

Inside the loop: `if (SUPPORTED_CURRENCIES.contains(currencyCode))` is the guard that makes unsupported currencies cost zero API calls. Only when this is true does the method call `ExchangeRateApiClient.getConversionRate(BASE_CURRENCY, currencyCode)` — the ONLY line in this

entire class that reaches out to the network (indirectly, through that client class). The result, success or failure, is stored in `rateCache` keyed by currency, and `outcome.apiCallCount` is incremented exactly once per actual call made.

Currencies NOT in `SUPPORTED_CURRENCIES` simply never get an entry written into `rateCache` at all for this run — which matters in the very next section, because `rateCache.get(currencyCode)` for an unsupported currency returns null, and the code downstream is written to handle that correctly (Section 3.7).

Execution Trace — Why this loop must finish completely before the next one starts

Salesforce enforces that ALL callouts in a transaction happen before ANY DML statement in that same transaction — attempting a DML statement after a callout has already occurred throws an immediate runtime error. This is precisely why the code is structured as two separate loops over `byCurrency.keySet()` rather than one combined loop that calls out AND writes a record in the same pass: the first loop (this one) does every single callout the whole method will ever make; only after it fully completes does the second loop (Section 3.6-3.7) begin building records that will eventually be written to the database.

3.6 The second loop — building the `Exchange_Rate_History__c` records

This is a SECOND, separate for (`String currencyCode : byCurrency.keySet()`) loop — same keys as before, but now happening entirely in memory with no further callouts.

- `ExchangeRateApiClient.RateResult rate = rateCache.get(currencyCode);` retrieves whatever was cached in Section 3.5 — this will be null for any currency that was never in `SUPPORTED_CURRENCIES`.
- `if (SUPPORTED_CURRENCIES.contains(currencyCode))` guards the entire history-record-building block, which is why NO `Exchange_Rate_History__c` record is ever created for an unsupported currency — there is nothing meaningful to log about a rate that was never even requested.
- Inside that guard, a new `Exchange_Rate_History__c()` is populated with `Base_Currency__c`, `Target_Currency__c`, and `Retrieved_Date_Time__c` unconditionally, and then EITHER `API_Status__c = 'Success'` plus `Exchange_Rate__c` (if `rate.isSuccess`) OR `API_Status__c = 'Failed'` plus `Error_Details__c` (otherwise) — never both.
- `rateHistoryRecords.add(historyRec);` appends to a List declared before either loop started; nothing is inserted into the database yet.

3.7 Still inside the second loop — the inner loop over Opportunities

Nested inside the currency-keyed loop is for (`Opportunity opp : byCurrency.get(currencyCode)`) — this is where every individual Opportunity that shares this currency finally gets evaluated, one at a time, entirely in memory.

- `Opportunity oppUpdate = new Opportunity(Id = opp.Id);` creates a NEW, mostly-empty Opportunity object carrying only the Id of the record to update. This is a standard and important Apex pattern: you never need to re-supply every field on a record just to update a few of them — `Database.update` only touches the fields you explicitly set on this sparse object, plus the Id telling it which row to update.
- `Decimal previousAmount = opp.Expected_Currency_Amount__c;` captures whatever the field's value was BEFORE this run, from the ORIGINAL queried opp, not from the new sparse

oppUpdate (which has no such value set). This captured value is what eventually becomes Previous_Expected_Currency_Amount__c on the tracking record.

The four-way if/else chain — the priority order in code

This is the exact sequence described narratively in the Technical Report, now traced as actual branches:

- if (opp.Amount == null) — checked FIRST, regardless of what the currency is or whether the API call for that currency even succeeded. Sets Failed + a specific "Amount is blank" message, increments failedCount, and the chain stops here for this record — none of the later branches run.
- else if (!SUPPORTED_CURRENCIES.contains(currencyCode)) — only reached if Amount was NOT null. Sets Failed + a message naming the specific unsupported currency AND listing the six that ARE supported, increments failedCount.
- else if (!rate.isSuccess) — only reached if the currency WAS supported, meaning rate is guaranteed non-null here (because the guard in 3.5 only skips caching for unsupported currencies) — but the actual API call for that supported currency still failed (invalid key, API outage, etc.). Sets Failed + whatever errorMessage the API client produced, increments failedCount.
- else — the only remaining possibility: Amount is present, the currency is supported, AND the API call for it succeeded. This is the sole branch that calculates a result and builds a tracking record.

Inside the success branch

- Decimal newAmount = (opp.Amount * rate.conversionRate).setScale(2, System.RoundingMode.HALF_UP); — the entire business formula of this whole project, in one line: multiply, then round to 2 decimal places using standard financial rounding.
- Five fields are set on oppUpdate: the new amount, the rate used, a fresh timestamp, Status = 'Success', and the error message field explicitly cleared to null (important: if a PREVIOUS run had left an error message on this record, a later successful run must actively erase it, or a stale error would sit next to a now-successful conversion).
- A new Opportunity_Currency_Update__c is built entirely from values already in memory — opp.Id, opp.Amount, the currency, the rate, previousAmount (captured a few lines earlier), and newAmount (just calculated) — and added to currencyUpdateRecords.

Regardless of which of the four branches executed, the very last line of the inner loop is unconditional: oppsToUpdate.add(oppUpdate); — every single Opportunity that entered this method, except the ones skipped for a blank currency back in Section 3.4, ends up in this list exactly once, whether its outcome was Success or Failed.

3.8 The final three DML statements

After both loops have fully completed — meaning every record has been fully evaluated in memory and every list is completely built — exactly three DML operations run, each guarded by an isEmpty() check so an empty list never triggers a pointless DML call:

- Database.insert(rateHistoryRecords, false); — writes the currency-level audit trail.
- Database.update(oppsToUpdate, false); — writes every Opportunity's new Status/Amount/Error fields, success or failure alike, in one bulk call.

- `Database.insert(currencyUpdateRecords, false);` — writes the record-level audit trail, only for the subset that actually succeeded.

The `false` argument to `each` is the `allOrNone` parameter: passing `false` means every record in that list is attempted independently, and one record failing validation does not prevent any of the others in the same list from being saved. See Section 6 (Testing) for how this is verified, and the Technical Report Section 7.5 for the general reasoning behind this choice.

Finally, `return outcome;` hands the four counters back to whichever caller invoked this method — `CurrencyConversionBatch.execute()` or `CurrencyConversionQueueable.execute()` — neither of which needs to know anything about HOW those numbers were produced, only what they are.

4. CurrencyConversionBatch — Line by Line

This class implements three interfaces at once: `Database.Batchable<SObject>`, `Database.Stateful`, and `Database.AllowsCallouts`. Each of the three required methods (start, execute, finish) runs at a different, specific point in the batch job's lifecycle.

```
public with sharing class CurrencyConversionBatch implements Database.Batchable<SObject>,
Database.Stateful, Database.AllowsCallouts {
```

```
    private String queryFilter;
```

```
    public Integer totalSuccess = 0;
```

```
    public Integer totalFailed = 0;
```

```
    public Integer totalSkipped = 0;
```

```
    public Integer totalApiCalls = 0;
```

```
    public CurrencyConversionBatch() {
```

```
        this.queryFilter = null;
```

```
    }
```

```
    public CurrencyConversionBatch(String queryFilter) {
```

```
        this.queryFilter = queryFilter;
```

```
    }
```

```
    public Database.QueryLocator start(Database.BatchableContext bc) {
```

```
        String baseQuery = 'SELECT Id, Amount, Expected_Currency_Type__c,
Expected_Currency_Amount__c, ' +
```

```
        'Currency_Update_Status__c FROM Opportunity';
```

```
        if (String.isNotBlank(queryFilter)) {
```

```
            baseQuery += ' WHERE ' + queryFilter;
```

```
        }
```

```
        return Database.getQueryLocator(baseQuery);
```

```
    }
```

```
    public void execute(Database.BatchableContext bc, List<Opportunity> scope) {
```

```
        CurrencyConversionService.ConversionOutcome outcome =
```

```
CurrencyConversionService.processOpportunities(scope);
```

```
        totalSuccess += outcome.successCount;
```

```
        totalFailed += outcome.failedCount;
```

```
        totalSkipped += outcome.skippedCount;
```

```
        totalApiCalls += outcome.apiCallCount;
```

```
    }
```

```
    public void finish(Database.BatchableContext bc) {
```

```
        System.debug('=== Currency Conversion Batch Complete ===');
```

```
        System.debug('Success: ' + totalSuccess);
```

```
        System.debug('Failed: ' + totalFailed);
```

```
        System.debug('Skipped: ' + totalSkipped);
```

```
        System.debug('Total API Calls: ' + totalApiCalls);
```

```
}  
}
```

4.1 The two constructors

`public CurrencyConversionBatch()` sets `queryFilter` to null. `public CurrencyConversionBatch(String queryFilter)` sets it to whatever was passed in. Both constructors do nothing else — the actual use of `queryFilter` happens entirely inside `start()`. This is the whole mechanism behind REQ 10's reprocessing feature: calling `new CurrencyConversionBatch()` (no-arg) processes everything; calling `new CurrencyConversionBatch('Currency_Update_Status__c = \'Failed\')` processes only that subset. No other code in this class needs to know or care which constructor was used.

4.2 start() — runs exactly once, before any chunking happens

- A base SOQL string is built as a `String`, not written as an inline query — this is necessary because the `WHERE` clause is only known at runtime, from `queryFilter`.
- `if (String.isNotBlank(queryFilter)) { baseQuery += ' WHERE ' + queryFilter; }` appends the filter ONLY if one was actually provided — the no-arg constructor path leaves the query as a plain, unfiltered `SELECT ... FROM Opportunity`.
- `return Database.getQueryLocator(baseQuery);` is the return value Salesforce actually uses to know which records exist and how many chunks to create. A `QueryLocator`, unlike a plain `List<Opportunity>`, does not hold all matching records in memory at once — Salesforce streams through the underlying query result set internally, which is why this pattern scales to far more than 500 records without any heap-size concern.

4.3 execute() — runs once per chunk, in its own transaction

The entire method body is four lines: call `CurrencyConversionService.processOpportunities(scope)` with whatever chunk of records Salesforce handed this particular invocation, then add each of the four returned counters onto this batch instance's own running totals (`totalSuccess`, `totalFailed`, etc.).

Execution Trace — Why += and not = here

Because `Database.Stateful` is implemented on this class, `totalSuccess` and the other three instance variables are NOT reset between chunks — the SAME instance persists across every `execute()` call for this job. Using `+=` accumulates each chunk's contribution on top of whatever the previous chunks already added. Without `Database.Stateful`, this class would still compile and run without error, but `totalSuccess` would silently only ever reflect the LAST chunk processed, because a fresh instance (with `totalSuccess` starting back at 0) would be used for every single `execute()` call.

4.4 finish() — runs exactly once, after every chunk has completed

Four `System.debug()` calls print the final accumulated totals. This is the only place in the whole class where the OVERALL result of the entire job (all chunks combined) is available — inside `execute()`, you only ever see one chunk's contribution at a time.

4.5 Concrete execution trace for the real 500-record run

- `Database.executeBatch(new CurrencyConversionBatch(), 200);` is called.

- `start()` runs once, returning a `QueryLocator` matching all 500 Task 05 Opportunities (plus, harmlessly, the 53 unrelated pre-existing ones with a null `Expected_Currency_Type__c`, which `processOpportunities` skips as usual).
- Salesforce splits the ~553 total matching records into three chunks of at most 200 each: chunk 1 (200 records), chunk 2 (200 records), chunk 3 (~153 records).
- `execute()` runs three times, once per chunk, each time calling `processOpportunities()` fresh with just that chunk's records — meaning currency grouping and the rate cache (Section 3.5) only ever apply WITHIN one chunk, never across chunks (this is the documented per-chunk caching trade-off).
- After the third `execute()` call returns, `finish()` runs once, logging the sum of all three chunks' outcomes.

5. Trigger, Handler, and Queueable — Line by Line

Three separate files work together here, each with a narrow role, because a callout cannot be made directly from trigger context (Technical Report, Section 9).

5.1 OpportunityCurrencyTrigger.trigger

```
trigger OpportunityCurrencyTrigger on Opportunity (after insert, after update) {
    OpportunityCurrencyTriggerHandler.handle(Trigger.new, Trigger.oldMap);
}
```

trigger OpportunityCurrencyTrigger on Opportunity (after insert, after update) fires this code after the database has already committed the insert or update — Trigger.new is guaranteed to reflect the final, saved field values at this point. The single statement inside simply forwards Trigger.new (the list of records as they now exist) and Trigger.oldMap (a Map<Id, Opportunity> of how each record looked immediately before this transaction, or null on insert, since there is no "before" state for a brand-new record) to the handler class.

5.2 OpportunityCurrencyTriggerHandler.cls

```
public with sharing class OpportunityCurrencyTriggerHandler {

    public static Boolean bypassTrigger = false;

    public static void handle(List<Opportunity> newOpps, Map<Id, Opportunity> oldMap) {
        if (bypassTrigger) {
            return;
        }

        Set<Id> idsToProcess = new Set<Id>();

        for (Opportunity opp : newOpps) {
            if (String.isBlank(opp.Expected_Currency_Type__c) || opp.Amount == null) {
                continue;
            }

            if (oldMap == null) {
                idsToProcess.add(opp.Id);
            } else {
                Opportunity oldOpp = oldMap.get(opp.Id);
                Boolean amountChanged = opp.Amount != oldOpp.Amount;
                Boolean currencyChanged = opp.Expected_Currency_Type__c !=
oldOpp.Expected_Currency_Type__c;
                if (amountChanged || currencyChanged) {
                    idsToProcess.add(opp.Id);
                }
            }
        }

        if (!idsToProcess.isEmpty()) {
```

```

        System.enqueueJob(new CurrencyConversionQueueable(idsToProcess));
    }
}

```

- `public static Boolean bypassTrigger = false;` is a class-level (not instance-level) flag, meaning ANY code anywhere in the same transaction can set `OpportunityCurrencyTriggerHandler.bypassTrigger = true;` and every subsequent call to `handle()` in that same transaction will see it.
- `if (bypassTrigger) { return; }` is the very first line of `handle()` — when the flag is set, the method does absolutely nothing else, for every record in the batch, and control returns immediately to the trigger (and from there, back to whatever DML operation fired it).
- `Set<Id> idsToProcess = new Set<Id>();` collects Ids, not full Opportunity records — deliberately lightweight, since the Queueable that eventually processes these Ids will re-query fresh data anyway (Section 5.3).
- The `for (Opportunity opp : newOpps)` loop first skips any record with a blank currency or a null Amount via a single combined condition — these could never produce a meaningful conversion regardless of what changed.
- `if (oldMap == null)` means this is an INSERT (there is no "old" version to compare against) — if the record passed the blank/null check above, it unconditionally qualifies for processing.
- Otherwise (an UPDATE), `Opportunity oldOpp = oldMap.get(opp.Id);` retrieves the pre-transaction version of this specific record, and two Booleans compare the new and old values of Amount and Expected_Currency_Type__c respectively. Only if at least one of them actually changed does the Id get added to `idsToProcess` — a Stage change, a Name change, or any other field update on the same record does NOT add it.
- After the loop, `if (!idsToProcess.isEmpty()) { System.enqueueJob(new CurrencyConversionQueueable(idsToProcess)); }` fires exactly ONE Queueable job for the entire transaction, carrying every qualifying Id from potentially hundreds of records updated in a single bulk operation — never one job per record.

5.3 CurrencyConversionQueueable.cls

```

public with sharing class CurrencyConversionQueueable implements Queueable,
Database.AllowsCallouts {

    private Set<Id> opportunityIds;

    public CurrencyConversionQueueable(Set<Id> opportunityIds) {
        this.opportunityIds = opportunityIds;
    }

    public void execute(QueueableContext context) {
        List<Opportunity> opps = [
            SELECT Id, Amount, Expected_Currency_Type__c, Expected_Currency_Amount__c,
Currency_Update_Status__c
            FROM Opportunity
            WHERE Id IN :opportunityIds
        ];
    }
}

```

```

        if (!opps.isEmpty()) {
            CurrencyConversionService.processOpportunities(opps);
        }
    }
}

```

- implements Queueable, Database.AllowsCallouts — the second interface is mandatory here for exactly the same reason it is required on the batch class: without it, any callout inside execute() throws immediately.
- The constructor simply stores the Set<Id> it was built with.
- execute(QueueableContext context) opens with a fresh SOQL query, re-selecting the affected Opportunities by Id from the database — NOT reusing whatever data the trigger originally saw. By the time this method actually runs (asynchronously, potentially some time after the original transaction committed), re-querying guarantees the logic operates on the current, authoritative state of each record.
- if (!opps.isEmpty()) { CurrencyConversionService.processOpportunities(opps); } — the same defensive empty-check pattern used elsewhere, followed by the identical service call used by the batch path. The return value (a ConversionOutcome) is not captured here, since there is no batch-level running total to add it to in this single-shot context.

6. Asynchronous Apex — How the Pieces Actually Execute

This project uses two different asynchronous mechanisms — Batchable and Queueable — and understanding exactly when code inside each one actually runs, relative to the rest of the transaction, is the hardest part of this codebase to reason about just from reading the source top to bottom.

6.1 Synchronous vs. asynchronous, in this codebase specifically

- Everything inside `OpportunityCurrencyTriggerHandler.handle()` runs **SYNCHRONOUSLY**, as part of the same transaction as the insert/update that fired the trigger. If `handle()` threw an unhandled exception, the entire insert/update would fail and roll back.
- `System.enqueueJob(...)` itself is synchronous — it returns almost instantly, having done nothing more than register the job for later execution. The trigger's transaction then finishes and commits normally.
- The Queueable's own `execute()` method runs **LATER**, in a completely separate transaction, with its own fresh set of governor limits, at a time controlled by the Salesforce platform (typically within seconds under normal load, but not guaranteed to be immediate, and not something Apex code can control precisely).
- Similarly, `Database.executeBatch(...)` is a synchronous call that only **SCHEDULES** the job; the actual `start()/execute()/finish()` calls all happen later, in their own separate transactions, one per chunk for `execute()`.

Execution Trace — A common misconception this project deliberately avoids

It is tempting to think of `System.enqueueJob(new CurrencyConversionQueueable(ids))` as "calling a method" the way a normal Apex method call works — but it is not. The line does not run the Queueable's `execute()` method at all; it only tells Salesforce "please run this at some point soon, in its own transaction." Any code written immediately after that `enqueueJob()` line in `handle()` runs before the Queueable has done **ANYTHING** — this is why the trigger's own transaction never waits on, or knows the outcome of, the currency conversion it just kicked off.

6.2 Why callouts are blocked by default in async Apex, and how the marker interfaces unblock them

Both `CurrencyConversionBatch` and `CurrencyConversionQueueable` explicitly implement `Database.AllowsCallouts`. Without this interface present on the class, any attempt to make an HTTP callout from inside `execute()` throws `System.CalloutException: Callout from batch Apex not allowed` (or an equivalent message for Queueable) — a hard platform restriction that exists so a batch or queueable class cannot accidentally make unbounded external calls without the developer explicitly opting in.

6.3 Database.Stateful — surviving across chunk boundaries

`CurrencyConversionBatch` implements `Database.Stateful` specifically because it needs `totalSuccess`, `totalFailed`, `totalSkipped`, and `totalApiCalls` to accumulate correctly across **THREE** separate transactions (one per 200-record chunk, for the real 500-record dataset). Without this interface, Salesforce would construct a brand-new `CurrencyConversionBatch` instance for every single `execute()` call, meaning each chunk's totals would silently overwrite, not add to, the previous chunk's — and by the time `finish()` ran, only the **LAST** chunk's numbers would remain.

6.4 A worked trace: one Opportunity, from a user's edit to a Success status

- 1. A user opens an existing Opportunity in the Salesforce UI and changes Amount from 50,000 to 75,000, then clicks Save.
- 2. Salesforce commits the update to the database. OpportunityCurrencyTrigger fires, synchronously, as part of this same save operation.
- 3. The trigger calls OpportunityCurrencyTriggerHandler.handle(Trigger.new, Trigger.oldMap). Inside handle(), oldOpp.Amount (50,000) is compared against opp.Amount (75,000) — they differ, so amountChanged is true, and this record's Id is added to idsToProcess.
- 4. System.enqueueJob(new CurrencyConversionQueueable(idsToProcess)) is called with a Set containing this one Id. handle() returns. The trigger's transaction commits. From the user's point of view, the Save operation is already complete at this point — they see the new Amount immediately, with Currency_Update_Status__c still showing whatever it was before (likely still 'Success' from an earlier conversion, or 'Pending' if this is the record's first-ever qualifying change).
- 5. Moments later, Salesforce starts a new, separate transaction and runs CurrencyConversionQueueable.execute(). It re-queries the Opportunity fresh by Id, getting the CURRENT saved state (Amount = 75,000).
- 6. execute() calls CurrencyConversionService.processOpportunities() with a list containing just this one Opportunity.
- 7. Inside processOpportunities(): the FLS check passes; the record is grouped into a one-entry Map keyed by its currency; exactly one callout is made for that currency; the record passes the Amount-not-null check, the supported-currency check, and the API-success check; the new converted amount is calculated from the NEW 75,000 Amount; the Opportunity is updated with Status = Success and the new Expected_Currency_Amount__c; a new Opportunity_Currency_Update__c record is inserted recording the old converted amount as "Previous" and the new one as "New".
- 8. This second transaction commits. The user, if they refresh the record, now sees the updated Status, the new converted amount, and — if they check Related records — a new tracking record documenting exactly what changed and why.

7. Testing — How Each Test Actually Works

All four test classes share one non-negotiable rule: no test may make a real HTTP callout. Salesforce enforces this at the platform level regardless of coding style — attempting a real callout in test context throws immediately. Every test that exercises callout-dependent code therefore implements `HttpCalloutMock` and registers it via `Test.setMock()` before triggering the code under test.

7.1 `ExchangeRateApiClientTest` — five independent scenarios

```
@isTest
private class ExchangeRateApiClientTest {

    private class SuccessMock implements HttpCalloutMock {
        public HTTPResponse respond(HTTPRequest req) {
            HttpResponse res = new HttpResponse();
            res.setStatusCode(200);

res.setBody('{ "result": "success", "base_code": "BDT", "target_code": "USD", "conversion_rate": 0.0082 }'
);
            return res;
        }
    }

    private class UnsupportedCodeMock implements HttpCalloutMock {
        public HTTPResponse respond(HTTPRequest req) {
            HttpResponse res = new HttpResponse();
            res.setStatusCode(403);
            res.setBody('{ "result": "error", "error-type": "unsupported-code" }');
            return res;
        }
    }

    private class InvalidKeyMock implements HttpCalloutMock {
        public HTTPResponse respond(HTTPRequest req) {
            HttpResponse res = new HttpResponse();
            res.setStatusCode(403);
            res.setBody('{ "result": "error", "error-type": "invalid-key" }');
            return res;
        }
    }

    private class MalformedJsonMock implements HttpCalloutMock {
        public HTTPResponse respond(HTTPRequest req) {
            HttpResponse res = new HttpResponse();
            res.setStatusCode(200);
            res.setBody('{not valid json}');
            return res;
        }
    }
}
```

```

    @isTest
    static void testSuccessfulConversion() {
        Test.setMock(HttpCalloutMock.class, new SuccessMock());

        Test.startTest();
        ExchangeRateApiClient.RateResult result = ExchangeRateApiClient.getConversionRate('BDT',
'USD');
        Test.stopTest();

        System.assertEquals(true, result.isSuccess, 'Expected success for a valid pair');
        System.assertEquals(0.0082, result.conversionRate, 'Conversion rate should match mock
response');
        System.assertEquals(200, result.httpStatusCode);
    }

    @isTest
    static void testUnsupportedCurrencyCode() {
        Test.setMock(HttpCalloutMock.class, new UnsupportedCodeMock());

        Test.startTest();
        ExchangeRateApiClient.RateResult result = ExchangeRateApiClient.getConversionRate('BDT',
'SGD');
        Test.stopTest();

        System.assertEquals(false, result.isSuccess, 'Unsupported code should not be a success');
        System.assertEquals('unsupported-code', result.errorType);
    }

    @isTest
    static void testInvalidKey() {
        Test.setMock(HttpCalloutMock.class, new InvalidKeyMock());

        Test.startTest();
        ExchangeRateApiClient.RateResult result = ExchangeRateApiClient.getConversionRate('BDT',
'USD');
        Test.stopTest();

        System.assertEquals(false, result.isSuccess);
        System.assertEquals('invalid-key', result.errorType);
    }

    @isTest
    static void testMalformedJsonResponse() {
        Test.setMock(HttpCalloutMock.class, new MalformedJsonMock());

        Test.startTest();
        ExchangeRateApiClient.RateResult result = ExchangeRateApiClient.getConversionRate('BDT',
'USD');

```

```

        Test.stopTest();

        System.assertEquals(false, result.isSuccess);
        System.assertEquals('parse-error', result.errorType);
    }

    @isTest
    static void testBlankCurrencyInput() {
        Test.startTest();
        ExchangeRateApiClient.Result result = ExchangeRateApiClient.getConversionRate('',
'USD');
        Test.stopTest();

        System.assertEquals(false, result.isSuccess, 'Blank base currency should fail fast
without a callout');
        System.assertEquals('invalid-input', result.errorType);
    }
}

```

- Four separate private classes (SuccessMock, UnsupportedCodeMock, InvalidKeyMock, MalformedJsonMock) each implement HttpCalloutMock once, each hardcoding exactly one canned HttpResponse. This is deliberately simple: each test method needs the API to behave one specific way, so each gets its own tiny, single-purpose mock rather than one complicated mock trying to branch on conditions.
- testSuccessfulConversion registers SuccessMock, calls getConversionRate('BDT', 'USD'), and asserts isSuccess is true AND the exact conversionRate value (0.0082) matches what the mock returned — proving the JSON-to-Decimal parsing chain (Section 2.6) works correctly end to end, not just that SOME value came back.
- testUnsupportedCurrencyCode and testInvalidKey follow the identical pattern with their respective mocks, asserting isSuccess is false and errorType matches exactly what the mock's JSON body specified.
- testMalformedJsonResponse deliberately returns a response body that is not valid JSON ('{not valid json}') to exercise the try/catch around JSON.deserializeUntyped described in Section 2.5, asserting errorType == 'parse-error'.
- testBlankCurrencyInput is the only method with NO Test.setMock call at all — passing an empty string as the base currency, and asserting the method returns a failed result with errorType == 'invalid-input' WITHOUT needing a mock, which is only possible because the blank-input guard (Section 2.2) returns before any callout is attempted. If that guard were ever accidentally removed, this test would immediately start failing with a "no mock callout response" error, making it also an implicit regression test for that guard's existence.

7.2 CurrencyConversionServiceTest — one @TestSetup, one comprehensive test method

```

@isTest
private class CurrencyConversionServiceTest {

    private class MultiCurrencyMock implements HttpCalloutMock {

```

```

    public HTTPResponse respond(HTTPRequest req) {
        HttpResponse res = new HttpResponse();
        String endpoint = req.getEndpoint();
        if (endpoint.endsWith('/USD')) {
            res.setStatusCode(200);
            res.setBody('{"result":"success","conversion_rate":0.0082}');
        } else if (endpoint.endsWith('/EUR')) {
            res.setStatusCode(200);
            res.setBody('{"result":"success","conversion_rate":0.0076}');
        } else {
            res.setStatusCode(403);
            res.setBody('{"result":"error","error-type":"unsupported-code"}');
        }
        return res;
    }
}

@TestSetup
static void setup() {
    OpportunityCurrencyTriggerHandler.bypassTrigger = true;

    List<Opportunity> opps = new List<Opportunity>();
    opps.add(new Opportunity(Name = 'USD Opp', StageName = 'Prospecting',
        CloseDate = Date.today().addDays(30), Amount = 100000, Expected_Currency_Type__c =
'USD'));
    opps.add(new Opportunity(Name = 'EUR Opp', StageName = 'Prospecting',
        CloseDate = Date.today().addDays(30), Amount = 200000, Expected_Currency_Type__c =
'EUR'));
    opps.add(new Opportunity(Name = 'Unsupported Opp', StageName = 'Prospecting',
        CloseDate = Date.today().addDays(30), Amount = 50000, Expected_Currency_Type__c =
'SGD'));
    opps.add(new Opportunity(Name = 'Blank Amount Opp', StageName = 'Prospecting',
        CloseDate = Date.today().addDays(30), Amount = null, Expected_Currency_Type__c =
'USD'));
    opps.add(new Opportunity(Name = 'Blank Currency Opp', StageName = 'Prospecting',
        CloseDate = Date.today().addDays(30), Amount = 75000, Expected_Currency_Type__c =
null));
    insert opps;

    OpportunityCurrencyTriggerHandler.bypassTrigger = false;
}

@isTest
static void testFullConversionBatch() {
    Test.setMock(HttpCalloutMock.class, new MultiCurrencyMock());
    List<Opportunity> opps = [SELECT Id, Amount, Expected_Currency_Type__c,
Expected_Currency_Amount__c
                            FROM Opportunity];

```

```

    Test.startTest();
    CurrencyConversionService.ConversionOutcome outcome =
CurrencyConversionService.processOpportunities(opps);
    Test.stopTest();

    System.assertEquals(2, outcome.successCount, 'USD and EUR opps should succeed');
    System.assertEquals(2, outcome.failedCount, 'Unsupported currency and blank amount should
fail');
    System.assertEquals(1, outcome.skippedCount, 'Blank currency should be skipped');
    System.assertEquals(2, outcome.apiCallCount, 'Should call API once each for USD and EUR
only; SGD is skipped since it is outside the supported currency list');

    Opportunity usdOpp = [SELECT Expected_Currency_Amount__c, Currency_Update_Status__c
                        FROM Opportunity WHERE Name = 'USD Opp'];
    System.assertEquals('Success', usdOpp.Currency_Update_Status__c);
    System.assertEquals(820.00, usdOpp.Expected_Currency_Amount__c);

    Opportunity blankCurrencyOpp = [SELECT Currency_Update_Status__c
                                   FROM Opportunity WHERE Name = 'Blank Currency Opp'];
    System.assertEquals('Pending', blankCurrencyOpp.Currency_Update_Status__c, 'Skipped opp
should remain at default Pending status');

    List<Exchange_Rate_History__c> historyRecs = [SELECT Target_Currency__c, API_Status__c
                                                FROM Exchange_Rate_History__c];
    System.assertEquals(2, historyRecs.size(), 'One history record per supported currency
actually queried; unsupported currencies are never queried');

    List<Opportunity_Currency_Update__c> trackingRecs = [SELECT Id FROM
Opportunity_Currency_Update__c];
    System.assertEquals(2, trackingRecs.size(), 'One tracking record per successful
conversion');
}
}

```

- @TestSetup static void setup() runs ONCE for the whole class (not once per test method), creating five Opportunities covering five distinct scenarios in one shot: a USD case, a EUR case, an unsupported-currency (SGD) case, a blank-Amount case, and a blank-currency case.
- OpportunityCurrencyTriggerHandler.bypassTrigger = true; wraps the insert statement specifically so that the trigger (which now exists on Opportunity) does not fire its own, separate, unmocked callout during this data setup — the flag is set back to false immediately afterward so it does not affect anything else running later.
- MultiCurrencyMock branches its response based on req.getEndpoint() — checking whether the URL ends in '/USD' or '/EUR' to return different rates for each, and falling back to an unsupported-code error for anything else (which, given the service's own SUPPORTED_CURRENCIES guard, is never actually reached for SGD in practice, since SGD never triggers a callout at all — the mock's SGD branch exists mainly for completeness/robustness).
- The single test method testFullConversionBatch re-queries all five Opportunities, calls processOpportunities() once with all of them together, and then asserts on FOUR different things: the ConversionOutcome counters (2 success, 2 failed, 1 skipped, 2 API calls), the exact

converted amount on the USD record (820.00, proving the formula and rounding), the skipped record's status ('Pending', not null — see Technical Report Section 5.1), and the COUNT of both tracking-object types created (2 history records, since only 2 distinct currencies were ever actually queried; 2 tracking records, matching the 2 successes).

7.3 CurrencyConversionBatchTest — proving the batch wrapper itself, not the logic again

```
@isTest
private class CurrencyConversionBatchTest {

    private class BatchMultiCurrencyMock implements HttpCalloutMock {
        public HTTPResponse respond(HTTPRequest req) {
            HTTPResponse res = new HTTPResponse();
            String endpoint = req.getEndpoint();
            if (endpoint.endsWith('/USD')) {
                res.setStatusCode(200);
                res.setBody('{"result":"success","conversion_rate":0.0082}');
            } else if (endpoint.endsWith('/EUR')) {
                res.setStatusCode(200);
                res.setBody('{"result":"success","conversion_rate":0.0076}');
            } else {
                res.setStatusCode(403);
                res.setBody('{"result":"error","error-type":"unsupported-code"}');
            }
            return res;
        }
    }

    @TestSetup
    static void setup() {
        OpportunityCurrencyTriggerHandler.bypassTrigger = true;

        List<Opportunity> opps = new List<Opportunity>();
        for (Integer i = 0; i < 50; i++) {
            opps.add(new Opportunity(
                Name = 'Batch Test Opp ' + i,
                StageName = 'Prospecting',
                CloseDate = Date.today().addDays(30),
                Amount = 10000 + i,
                Expected_Currency_Type__c = (Math.mod(i, 2) == 0) ? 'USD' : 'EUR'
            ));
        }
        opps.add(new Opportunity(Name = 'Unsupported Batch Opp', StageName = 'Prospecting',
            CloseDate = Date.today().addDays(30), Amount = 5000, Expected_Currency_Type__c =
'SGD'));
        insert opps;

        OpportunityCurrencyTriggerHandler.bypassTrigger = false;
    }
}
```

```

}

@isTest
static void testFullBatchRun() {
    Test.setMock(HttpCalloutMock.class, new BatchMultiCurrencyMock());

    Test.startTest();
    CurrencyConversionBatch batch = new CurrencyConversionBatch();
    Database.executeBatch(batch, 200);
    Test.stopTest();

    List<Opportunity> updated = [SELECT Currency_Update_Status__c FROM Opportunity
                               WHERE Expected_Currency_Type__c IN ('USD', 'EUR')];
    for (Opportunity opp : updated) {
        System.assertEquals('Success', opp.Currency_Update_Status__c);
    }

    Opportunity unsupportedOpp = [SELECT Currency_Update_Status__c FROM Opportunity
                                  WHERE Name = 'Unsupported Batch Opp'];
    System.assertEquals('Failed', unsupportedOpp.Currency_Update_Status__c);
}

@isTest
static void testReprocessFilteredBatch() {
    Test.setMock(HttpCalloutMock.class, new BatchMultiCurrencyMock());

    Test.startTest();
    Database.executeBatch(new CurrencyConversionBatch(), 200);
    Test.stopTest();

    List<Opportunity> failedOpps = [SELECT Id FROM Opportunity WHERE
Currency_Update_Status__c = 'Failed'];
    System.assertEquals(1, failedOpps.size(), 'Only the unsupported currency opp should be
Failed');
}
}

```

- @TestSetup creates 51 Opportunities: 50 alternating between USD and EUR (using Math.mod(i, 2) to alternate), plus one SGD outlier — again wrapped in the bypassTrigger flag.
- testFullBatchRun wraps Database.executeBatch(batch, 200) inside Test.startTest()/Test.stopTest() — this is what forces the batch's start(), execute(), and finish() to actually run, synchronously, before the test method continues past Test.stopTest(). The batch size of 200 (larger than the 51 test records) ensures everything fits in a single chunk, which is REQUIRED inside test context (Section 7.5).
- After stopTest(), the test queries every USD/EUR Opportunity and asserts each one individually has Status == 'Success' in a loop — proving the batch correctly processed the FULL set, not just a sample of it.
- testReprocessFilteredBatch runs the same no-argument batch and then asserts exactly ONE record has Status == 'Failed' (the single SGD outlier) — a lightweight sanity check on the filtering

logic's correctness at the record-count level, complementing the more detailed assertions in `CurrencyConversionServiceTest`.

7.4 OpportunityCurrencyTriggerTest — proving the trigger fires (and doesn't) correctly

```
@isTest
private class OpportunityCurrencyTriggerTest {

    private class TriggerMock implements HttpCalloutMock {
        public HTTPResponse respond(HTTPRequest req) {
            HTTPResponse res = new HTTPResponse();
            res.setStatusCode(200);
            res.setBody('{"result":"success","conversion_rate":0.0082}');
            return res;
        }
    }

    @isTest
    static void testInsertTriggersConversion() {
        Test.setMock(HttpCalloutMock.class, new TriggerMock());

        Test.startTest();
        Opportunity opp = new Opportunity(
            Name = 'Trigger Test Opp',
            StageName = 'Prospecting',
            CloseDate = Date.today().addDays(30),
            Amount = 100000,
            Expected_Currency_Type__c = 'USD'
        );
        insert opp;
        Test.stopTest();

        Opportunity result = [SELECT Currency_Update_Status__c, Expected_Currency_Amount__c
                               FROM Opportunity WHERE Id = :opp.Id];
        System.assertEquals('Success', result.Currency_Update_Status__c);
        System.assertEquals(820.00, result.Expected_Currency_Amount__c);
    }

    @isTest
    static void testUpdateAmountTriggersConversion() {
        Test.setMock(HttpCalloutMock.class, new TriggerMock());

        Opportunity opp = new Opportunity(
            Name = 'Trigger Update Opp',
            StageName = 'Prospecting',
            CloseDate = Date.today().addDays(30),
            Amount = 50000,
            Expected_Currency_Type__c = 'USD'
        );
```

```

    );
    insert opp;

    Test.startTest();
    opp.Amount = 75000;
    update opp;
    Test.stopTest();

    Opportunity result = [SELECT Currency_Update_Status__c FROM Opportunity WHERE Id
= :opp.Id];
    System.assertEquals('Success', result.Currency_Update_Status__c);
}

@isTest
static void testUnrelatedFieldUpdateDoesNotTrigger() {
    Test.setMock(HttpCalloutMock.class, new TriggerMock());

    Opportunity opp = new Opportunity(
        Name = 'No Trigger Opp',
        StageName = 'Prospecting',
        CloseDate = Date.today().addDays(30),
        Amount = 30000,
        Expected_Currency_Type__c = 'USD'
    );
    insert opp;

    Test.startTest();
    opp.StageName = 'Closed Won';
    update opp;
    Test.stopTest();

    Opportunity result = [SELECT Currency_Update_Status__c FROM Opportunity WHERE Id
= :opp.Id];
    System.assertEquals('Success', result.Currency_Update_Status__c,
        'The initial insert triggers conversion; the later StageName-only update should not
trigger a second one, but status remains Success from the insert');
}

@isTest
static void testBypassFlagPreventsProcessing() {
    OpportunityCurrencyTriggerHandler.bypassTrigger = true;

    Test.startTest();
    Opportunity opp = new Opportunity(
        Name = 'Bypassed Opp',
        StageName = 'Prospecting',
        CloseDate = Date.today().addDays(30),
        Amount = 40000,
        Expected_Currency_Type__c = 'USD'
    );

```

```

    );
    insert opp;
    Test.stopTest();

    OpportunityCurrencyTriggerHandler.bypassTrigger = false;

    Opportunity result = [SELECT Currency_Update_Status__c FROM Opportunity WHERE Id
= :opp.Id];
    System.assertEquals('Pending', result.Currency_Update_Status__c,
        'Bypassed insert should stay at default Pending with no callout made');
    }
}

```

- testInsertTriggersConversion wraps a plain insert (not update) inside Test.startTest()/stopTest(), then asserts the resulting Status is Success and the converted amount is correct — proving the ENTIRE chain (trigger fires -> handler decides to enqueue -> Queueable runs -> service processes) works correctly starting from nothing but a normal DML insert, with no direct call to any of those classes' methods anywhere in the test itself.
- testUpdateAmountTriggersConversion inserts a record BEFORE Test.startTest() (so that initial insert's own triggered conversion is not what is being measured), then changes Amount and updates it INSIDE Test.startTest()/stopTest(), asserting the SECOND conversion (triggered by the amount change) also succeeds.
- testUnrelatedFieldUpdateDoesNotTrigger inserts a record (which itself triggers a conversion, correctly mocked), then changes only StageName and updates — asserting the Status remains Success from the INITIAL insert's conversion, which is the test's way of confirming a second, unwanted conversion was never triggered by the unrelated field change (if it had been, and had failed for any reason, this assertion would likely show a different status or the test would show unexpected coverage/behavior elsewhere).
- testBypassFlagPreventsProcessing sets bypassTrigger = true before an insert, asserts Status stays at the default 'Pending' (proving no callout was ever attempted — this test method notably has NO Test.setMock call at all, which would cause an immediate failure if the bypass flag were broken and a real callout were attempted), then resets the flag to false so it cannot leak into any other test.

8. API Integration — Exactly How a Callout Resolves

This section traces one single callout from the literal string in the Apex code all the way to the real HTTP request that leaves Salesforce's servers, since the `callout:CredentialName/path` syntax hides several steps that are worth making explicit.

8.1 What the Apex code actually contains

```
req.setEndpoint('callout:ExchangeRateAPI/pair/BDT/USD');
```

This string contains no real hostname, no protocol, and no authentication information whatsoever. 'callout:' is a reserved prefix Salesforce recognizes; 'ExchangeRateAPI' is the exact API Name of the Named Credential record created in Setup; everything after the slash ('/pair/BDT/USD') is a literal path suffix that gets appended, unmodified, to whatever base URL that Named Credential record specifies.

8.2 What Salesforce does before the request ever leaves the platform

- Salesforce looks up the Named Credential record named "ExchangeRateAPI" and reads its configured URL: `https://v6.exchangerate-api.com/v6`.
- It concatenates that base URL with the path suffix from the Apex code, producing the real, final URL: `https://v6.exchangerate-api.com/v6/pair/BDT/USD`.
- It follows the link from that Named Credential to its associated External Credential ("ExchangeRateAPI_Cred") to determine authentication.
- Because that External Credential defines a Custom Header (Name: "Authorization", Value: "Bearer <the real key>"), Salesforce attaches an `Authorization: Bearer <key>` header to the outgoing request automatically.
- Only NOW, with the real URL and the real header both fully assembled, does the actual outbound HTTPS request leave Salesforce's infrastructure.

Execution Trace — The security property this produces

At no point in this sequence does Apex code ever hold, read, reference, or have the ability to log the real API key — the substitution happens entirely inside Salesforce's platform layer, after the Apex transaction has already specified everything it needs to specify. This is why a Named Credential is considered secure even if someone were to read every line of this project's Apex source code, or every line of a debug log generated while it ran: the key simply never appears in either place.

8.3 The real response shape, and exactly which Apex lines touch which JSON keys

A successful response body from the pair-conversion endpoint looks like this (abbreviated):

```
{
  "result": "success",
  "base_code": "BDT",
  "target_code": "USD",
  "conversion_rate": 0.0082
}
```

- "result" is read by `String apiResult = (String) body.get('result');` and compared against the literal string 'success'.

- "conversion_rate" is read by `Object rateObj = body.get('conversion_rate');` and converted to `Decimal` via the `String` round-trip described in Section 2.6.
- "base_code" and "target_code" are present in the real response but are never read anywhere in this codebase — they are redundant with the base/target currency values the calling code already knows, since it supplied them in the request URL itself.

A failure response looks like this instead:

```
{
  "result": "error",
  "error-type": "unsupported-code"
}
```

Here, "result" is "error" (not "success"), so the if branch in Section 2.6 is not taken; instead, "error-type" is read via `(String) body.get('error-type')` and stored as `result.errorType`, which is exactly the value `CurrencyConversionService` later inspects (indirectly, via `rate.isSuccess` and the error message it builds) when deciding how to record this failure.

9. Quick Reference

9.1 Class responsibility, one line each

Class / Trigger	Responsibility
ExchangeRateApiClient	Makes the HTTP callout; parses the response; never touches Opportunity data
CurrencyConversionService	FLS check, grouping, validation, calculation, all DML — the one place business rules live
CurrencyConversionBatch	Drives processOpportunities() across chunks of an arbitrarily large record set
OpportunityCurrencyTrigger	One line: forwards Trigger.new/oldMap to the handler
OpportunityCurrencyTriggerHandler	Decides WHICH changed records qualify; enqueues one Queueable per transaction
CurrencyConversionQueueable	Re-queries qualifying records; calls processOpportunities() asynchronously

9.2 Who calls whom

```
Database.executeBatch(new CurrencyConversionBatch(), 200)
-> CurrencyConversionBatch.start()      [once]
-> CurrencyConversionBatch.execute()    [once per chunk]
    -> CurrencyConversionService.processOpportunities()
        -> ExchangeRateApiClient.getConversionRate() [once per distinct supported currency]
-> CurrencyConversionBatch.finish()     [once]

insert/update Opportunity
-> OpportunityCurrencyTrigger
    -> OpportunityCurrencyTriggerHandler.handle()
        -> System.enqueueJob(new CurrencyConversionQueueable(ids))
            [... separate async transaction ...]
        -> CurrencyConversionQueueable.execute()
            -> CurrencyConversionService.processOpportunities()
                -> ExchangeRateApiClient.getConversionRate()
```

9.3 The four-way failure/success priority order (inside processOpportunities)

1. Amount == null -> Failed: "Amount is blank"
2. currency NOT in SUPPORTED_CURRENCIES -> Failed: "Unsupported currency: X..."
3. currency supported, but API call failed -> Failed: <API's own error message>
4. currency supported AND API call succeeded -> Success: amount calculated & saved

9.4 Interfaces and what each one unlocks

Interface	Applied to	What it unlocks / enforces
Database.Batchable<SObject>	CurrencyConversionBatch	Enables start()/execute()/finish() chunked processing
Database.Stateful	CurrencyConversionBatch	Instance variables persist across chunks instead of resetting
Database.AllowsCallouts	CurrencyConversionBatch,	Permits HTTP callouts inside async

Interface	Applied to	What it unlocks / enforces
	CurrencyConversionQueueable	execute()
Queueable	CurrencyConversionQueueable	Enables asynchronous, chainable, callout-capable execution
HttpCalloutMock	4 private test-only classes	Intercepts callouts in tests; real callouts are never permitted in tests